

Key Points

Course: Social Network Analysis, UIT, VNUHCM
(aka. Graph and Social Media Analysis with Applications)

Lecturer: Dr. Tran Hung Nghiep
2025-2026

✓ Topic 2: Network Representation & Tools

1. Edge List Representation

An edge list stores the graph as a list of edges. For an undirected, unweighted graph:

$$E = \{(u_1, v_1), (u_2, v_2), \dots, (u_m, v_m)\}$$

For weighted edges, each entry may include weight:

$$(u_i, v_i, w_i)$$

Use set of tuples format when defining edges mathematically.

For example:

$$\{(A, B), (B, C), (C, A), (C, D)\}$$

Use plain column format when demonstrating file input.

For example:

```
A B
B C
C A
C D
```

Intuition

This is the simplest, most flexible format. Easy to load from text files, CSV, logs, etc.

Notes

- Good for **sparse** graphs.
- Fast to iterate over all edges, but slow for “is (u, v) an edge?” because it requires scanning or hashing.
- No quick access to neighbors unless converted to adjacency list.

2. Adjacency List Representation

Each node u stores the list of neighbors:

$$\Gamma(u) = \{v \mid (u, v) \in E\}$$

Formally, adjacency list is:

$$\text{AdjList} = \{(u, \Gamma(u)) : u \in V\}$$

Use set of adjacency tuples format when defining adjacency list mathematically.

For example:

$$\{(A, \{B, C\}), (B, \{A, C\}), (C, \{A, B, D\}), (D, \{C\})\}$$

Use plain column format when demonstrating file input.

For example:

```
A: B, C
B: A, C
C: A, B, D
D: C
```

Intuition

Most efficient for sparse networks. Many graph algorithms rely on fast neighbor iteration.

Notes

- Memory: $O(n + m)$
- Best for real networks where $m \ll n^2$
- Ideal for BFS/DFS, centrality algorithms, clustering coefficient computation, etc.

3. Adjacency Matrix Representation

Defines a matrix $A \in \mathbb{R}^{n \times n}$:

$$A_{ij} = \begin{cases} 1 & \text{if edge } (i, j) \text{ exists} \\ 0 & \text{otherwise} \end{cases}$$

For weighted graphs, entries are weights w_{ij} . For undirected graphs, A is symmetric.

Example

Nodes: A=1, B=2, C=3

Edges: (A,B), (B,C), (A,C)

Matrix:

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

Intuition

Very fast for checking the existence of an edge; supports linear algebra-based methods (spectral clustering, eigenvector centrality).

Notes

- Memory: $O(n^2) \rightarrow$ not feasible for large graphs.
- Necessary for some algorithms requiring matrix operations.

4. Comparing Three Representations

Operation	Edge List	Adjacency List	Adjacency Matrix
Memory	$O(m)$	$O(n + m)$	$O(n^2)$
Iterate neighbors	Slow	Fast	Fast (scan row)
Check if (u,v) exists	Slow	Moderate	Very fast (constant time)
Good for sparse graphs	Yes	Yes	No
Good for dense graphs	OK	OK	Yes
Good for matrix algebra	Poor	Poor	Excellent

Key Intuition

- Use adjacency **matrix** only if small or dense, or if spectral methods are required.
- Use adjacency **list** for compact storage and many algorithmic tasks.
- Use **edge list** for data input (especially ML models), storage, or logs.

5. Graph Data Input Formats in Practice

Common file formats:

- **Edge list** formats: `.txt`, `.tsv`, `.csv`
- **GML**: human-readable graph markup
- **GraphML**: XML-based graph standard
- **JSON**-like graph formats (e.g., in D3.js)
- **SNAP** format: used in Stanford CS224W datasets
- **NetworkX pickles**: for fast loading

Example (SNAP format)

Lines start with:

- `#` comments
- Each line: `u v` Nodes indexed with integers.

Notes

- Students should be comfortable loading real data from TXT/CSV.
- Pay attention to **indexing**: 0-indexed vs 1-indexed.

6. Graph Construction in NetworkX

Creating graphs:

```
import networkx as nx
G = nx.Graph() # undirected
G.add_edge('A', 'B')
G.add_edge('B', 'C')
```

Loading from file:

```
G = nx.read_edgelist("file.txt")
```

Directed graph:

```
DG = nx.DiGraph()  
DG.add_edge('A', 'B')
```

Weighted edges:

```
G.add_weighted_edges_from([('A', 'B', 3.5)])
```

Notes

- NetworkX automatically handles new nodes.
- Beware: NetworkX is not optimized for very large graphs (>1M edges).

7. Basic Graph Operations in NetworkX

Common operations:

- Node/edge counts:

```
G.number_of_nodes()  
G.number_of_edges()
```

- Degrees:

```
G.degree('A')  
dict(G.degree())
```

- Neighbors:

```
list(G.neighbors('A'))
```

- Connected components:

```
list(nx.connected_components(G))
```

- Simple visualization (matplotlib):

```
nx.draw(G, with_labels=True)
```

8. Graph Visualization Using Gephi

Gephi is a GUI tool for interactive visualization:

- Supports large graphs (~50k nodes or more).
- Algorithms included:
 - ForceAtlas2 layout
 - Modularity (Louvain)
 - Degree/centrality coloring
 - Statistics panel for distribution plots

Example Workflow

1. Load edge list into Gephi
2. Use layout (e.g., "ForceAtlas2")
3. Color nodes by degree or community
4. Size nodes by centrality
5. Export high-resolution image

Notes

- Visualization is exploratory, not formal analysis.
 - Avoid misleading layouts (e.g., interpreting proximity as similarity).
 - Gephi is good for manual visualization with GUI. For code-centric visualization, such as semantic trees, **igraph** is a good alternative.
-

9. Graph Learning Tools: Light Introduction

PyTorch Geometric (PyG) and DGL:

- Support end-to-end training of GNNs.
- Store graph as:
 - Edge index: a $2 \times m$ matrix $[u_i; v_i]$
 - Node features X
 - Edge features (optional)

Example edge index in PyG:

```
edge_index = torch.tensor([
    [0, 1, 1, 2],
    [1, 0, 2, 1]
], dtype=torch.long)
```

Notes

- Students should recognize basic syntax but details about GNN training will be learned later.
-

10. Data Structure Trade-Offs in Algorithms

Why representation choice matters:

- BFS/DFS $\rightarrow$ efficient with adjacency list
- Eigenvector/PageRank $\rightarrow$ adjacency matrix or CSR sparse matrix
- Link prediction heuristics $\rightarrow$ require neighbor lookups (adj list ideal)
- Spectral clustering $\rightarrow$ eigenvalue decomposition $\rightarrow$ matrix-based

Example

Shortest path BFS complexity:

- With adj list: $O(n + m)$
- With adj matrix: $O(n^2)$

Notes

Representation choice affects:

- Algorithm speed
 - Memory footprint
 - Feasibility of analysis on large networks
-

11. Typical Workflow in Coding Assignments

1. Load edge list from file
2. Convert to NetworkX graph
3. Explore basic info (n , m , degree stats)
4. Convert representation if needed (adj matrix, CSR matrix)
5. Visualize subgraph in Gephi
6. Prepare data for downstream tasks (centrality, community detection, link prediction)

Notes

- Students should be comfortable converting between representations and formats.
-

12. Common Pitfalls in Graph Representation

- Forgetting direction when loading digraphs $\rightarrow$ wrong centrality results
 - Treating weighted edges as unweighted $\rightarrow$ wrong shortest paths
 - Mistaking node labels (string vs int) $\rightarrow$ mismatch in matrix indices
 - Using adjacency matrix for huge sparse graphs $\rightarrow$ memory blow-up
 - Assuming neighbor order means something $\rightarrow$ it generally does NOT
-